

CS415/515 Milestone 1 Report

BenchTime: A lab equipment reservation system

Luke McArthur & Jesse Seidel

lgmcarthur@crimson.ua.edu & jpseidel@crimson.ua.edu

Submitted September 30th, 2026

Updated Product Brief and Scope

Introduction

The problem with many of The University of Alabama's laboratory and maker spaces used for ECE classes, such as the labs located in SERC, is that they lack a formal way of reserving lab equipment and space. The systems that are used in these classes are usually non-existent or extremely informal like using a sheet of paper to reserve time for equipment. Because of this, students often have to travel to a lab only to find out that the equipment they need is unavailable. The other issue is that without a good system for reserving equipment, lab managers/owners have no good way to know what equipment is commonly used so that they can make informed decisions about maintenance and future purchases.

Idea/Solution

Our solution to the problems mentioned above is the creation of BenchTime. BenchTime serves two primary types of users: lab managers and lab members. BenchTime allows managers to create their own labs(also known as equipment charts) where they can invite members and create reservable pieces of equipment. Members can be invited to multiple labs by different lab managers and within each lab reserve equipment. BenchTime operates on a first-come first-serve basis and automatically rejects any overlapping reservations. BenchTime also leaves all moderation up to the lab manager and does not provide any way of enforcing reservations in-person. In future milestones, BenchTime intends to allow managers to view utilization summaries about the pieces of equipment in their labs to assist them in making informed decisions.

MVP Scope

The features chosen for the MVP were those considered absolutely important for the basic operation of BenchTime. These features were chosen in milestone 0 and largely remained unchanged. Roughly the features currently available are as follows: member and manager account creation(enforced using JSON web tokens), allowing managers to create labs/equipment calendars, allowing managers to add members to a lab/equipment calendar(and this change appearing for the member), allowing managers to add equipment to a lab/equipment calendar, and allowing both the managers and members of a lab to reserve pieces of equipment for up to 6 hours. All data is stored in a SQL database and the application can currently be deployed locally and tested. Utilization statistics, testing via a CI/CD pipeline, deployment on a remote server, and more advanced reservation options(such as making a reservation repeat every week), were all considered outside of the MVP scope. Utilization statistics in particular were left out because while they are important for BenchTime, it was decided to focus more time/effort on the reservation aspect. While left out of the MVP, these features will be included in milestone 2 and further refined by milestone 3.

Chosen Software Process Model and Justification

Process Model

We used an agile process. For Milestone 1, we had two sprints from September 2-15 and September 16-29. The first sprint was mainly used for planning before actually building the project. We created a list of features that were needed for the MVP. This list was used to decide what features we would work on and in what order. For each feature, the developer would create a branch with that issue number at the start of the branch name, like '12-reserve-equipment'. This made it easy to keep the issues and branches in track. We used a KanBan board on Github Projects to track these issues. The board has columns: Todo, In Progress, and Done. This gives a place so that each team member knows what the other is working on and what is completed. With only two people on the team, it was easy to communicate and plan who was completing each feature.

Justification

The agile method is the correct process model since our requirements will continue to change throughout the project. Another reason an agile method is correct is because our team is small. The agile process is a not heavy process model. We can use a lightweight version that keeps the parts that work for us (backlog, peer reviews, sprints, KanBan board), while not using some unneeded parts that are better suited for bigger teams (daily standups, product owner, scrum masters).

Prioritized Backlog and Definition of Done

Backlog

The backlog can be found on the github repository [here](#). This backlog has been updated to include the progress made in milestone 1 and to include the features that will be added in milestone 2.

Definition of Done

This definition of done has been taken from milestone 0 and modified to reflect the changes we plan to make while working on milestone 2. Our definition of done is that a task is not finished until:

1. It does what the issue asked and we have tested it in a running version.
2. It has been reviewed and merged into main through a pull request. Since we are a group of two people, code review will be done by the other person. Any conflicts here will relay the task being finished until an agreement is reached.
3. It has tests, and the whole test suite still passes. “Passes” here means passing the automated testing via a CI/CD pipeline and not the manual testing done for milestone 1.
4. The app still runs from a fresh clone using the setup instructions in the README. This will be done using the CI/CD pipeline.

Use Cases

The following use cases are for the backlog items for milestone 2. The acceptance criteria for all items can be found [here](#).

Use Case(U-10): Manager edits reservations in their labs

Primary Actor: Lab/equipment chart manager(an authenticated manager).

Preconditions: The manager has already registered, and is logged into their account. They have also already navigated to a specific lab page.

Main Success Scenario

1. The manager selects a piece of equipment they would like to view the reservations for.
2. The manager sees a reservation attached to that equipment that they wish to edit.
3. The manager selects that reservation and an editing menu appears.
4. The manager makes the edit to the reservation.
5. The manager hits a button and the reservation edit is saved to a database and is now visible to all users.

Extensions

4a. The manager makes an invalid edit. For example, they change the start time of the reservation so that it overlaps with a previous one. Then the system shall display an error message to the manager telling them their new reservation time is invalid.

5a. There is an issue with saving the edit to the database. Then the system shall display an error saying that the manager's edit could not be saved and that they should try again later.

Use Case(U-11): Manager can view utilization statistics for a piece of equipment.

Primary Actor: Lab/equipment chart manager(an authenticated manager).

Preconditions: The manager has already registered, and is logged into their account. They have also already navigated to a specific lab page.

Main Success Scenario

1. The manager selects the piece of equipment they would like to view utilization statistics for.
2. They hit a button to view the utilization statistics.
3. The utilization statistics for that piece of equipment appear. This includes: how long the piece of equipment was reserved in a week, how often it was reserved, and when it was normally reserved.
4. After viewing the statistics, the manager hits another button to make them disappear.

Extensions

3a. No utilization statistics are available because the equipment was only just added. An error message shall appear telling the manager that more time must pass before they can view utilization statistics.

3b. The utilization statistics for that piece of equipment could not be retrieved from the database. An error message shall appear telling the manager that the statistics could not be found and that they should try again later.

Use Case (U-12) Member sets up recurring reservation

Primary Actor: Lab member

Preconditions: The member is logged into their account and has been added to the lab by a manager and is on the lab's page.

Main Success Scenario:

1. Member clicks the Reserve button next to the equipment.
2. The member enters a start date, stop date, a start time, and an end time.
3. The member toggles something to make it repeat every week.
4. The member submits the form.
5. The system creates a reservation every week at that time from start date to end date.

Extensions:

3a. The member picks a stop date that is too far out. The system will give an error.

5a. If one of the repeated reservations has a conflict, the whole save fails.

Use Case (U-13) Manager signs in using their university account (SSO)

Primary Actor: Lab manager or member

Preconditions: The user has a university email account and are on the BenchTime login page.

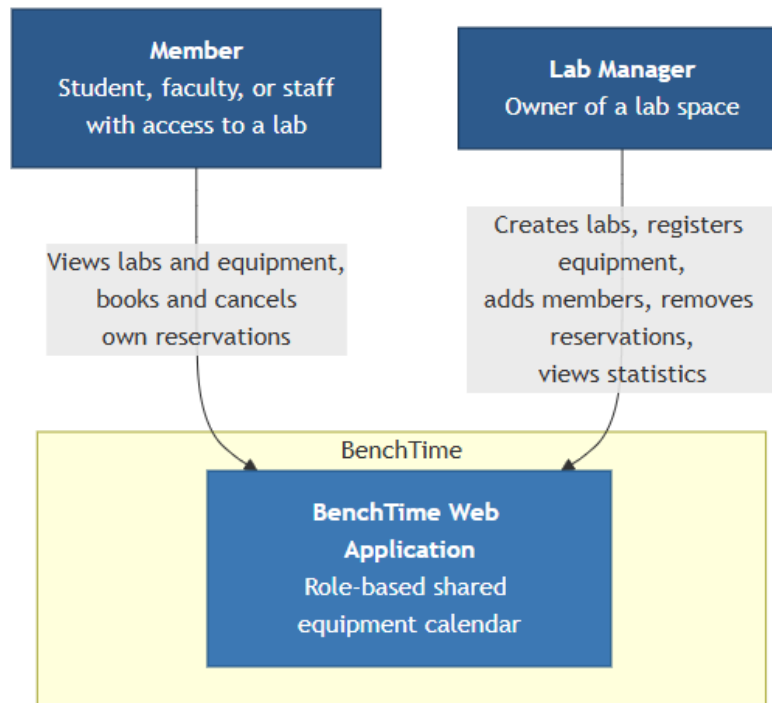
Main Success Scenario:

1. The user clicks a login button on the BenchTime login page.
2. The user is redirected to the university's login page.
3. The user enters their login information for the university login page.
4. The university's system verifies the user.
5. The user is automatically navigated back to BenchTime, logged in and on the home page.

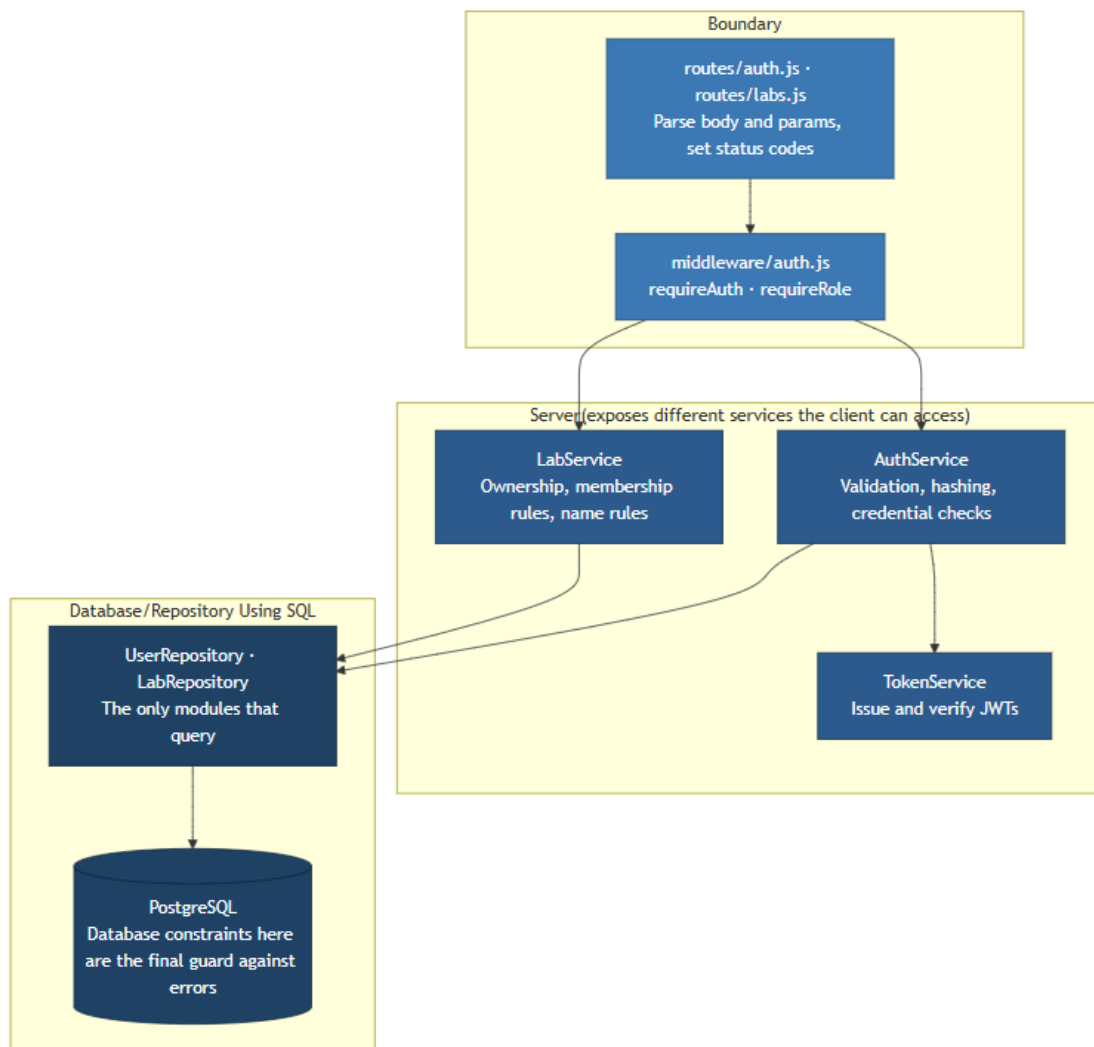
Extensions:

4a. The system denies access due to invalid login credentials. The university's system will alert the user that they are not logged in.

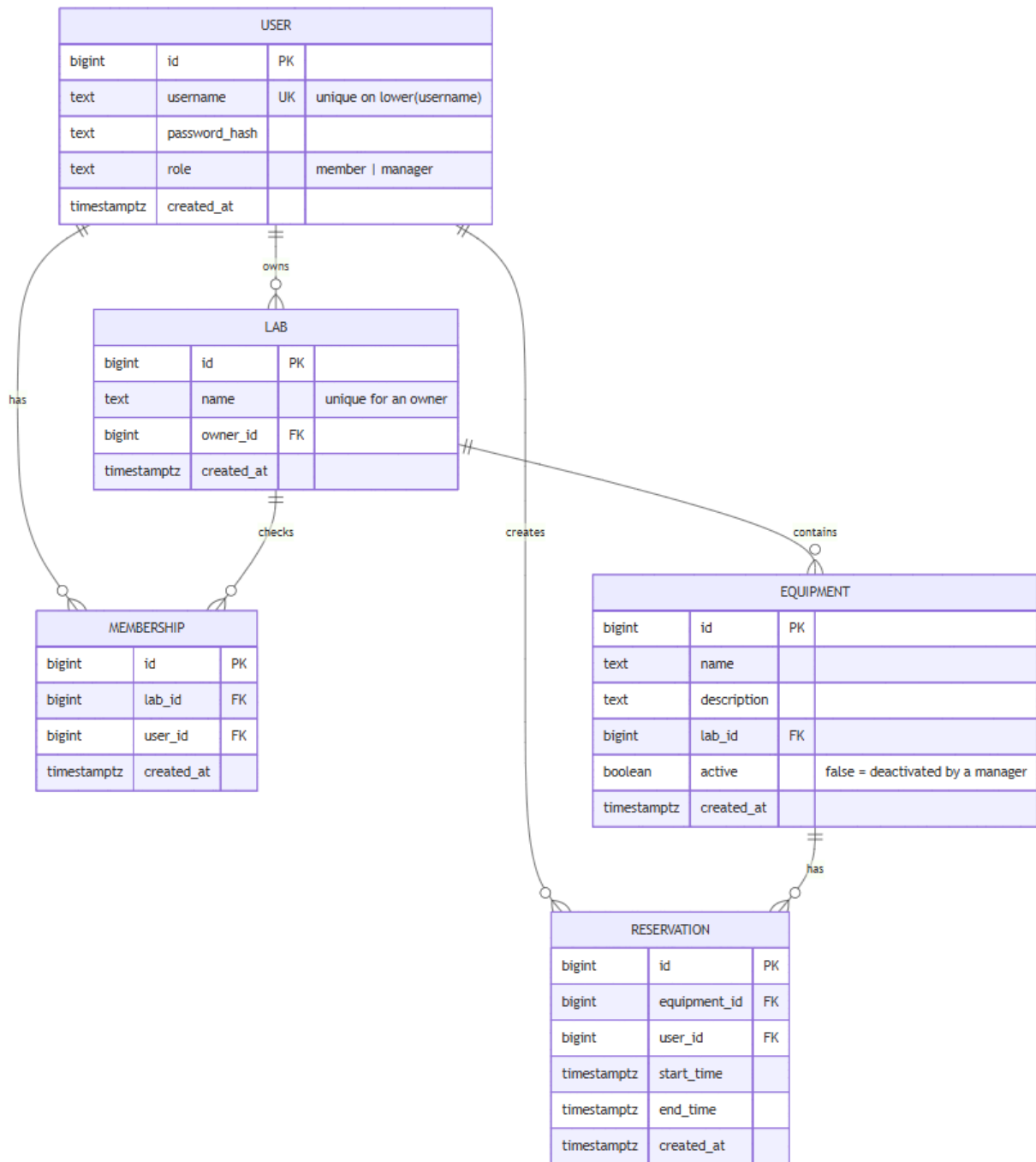
Architecture/Analysis Diagrams



The diagram above shows a very simple system context diagram for BenchTime. It gives a brief overview of both users and their responsibilities.



This diagram shows how BenchTime is actually structured. In general, the client(boundary) must only interact with the server via dedicated endpoints. These endpoints require authentication using JWT. The server can only interact with the database via a repository layer that is responsible for formatting queries and querying the database. The dataflow here is essentially: client -> server -> database/repository then database/repository -> server -> client.



This diagram shows how the data is actually stored inside of the BenchTime database. It is essentially a visual representation of the tables created inside of SQL. At the end of milestone 2, this diagram will be updated to reflect the insertion of utilization statistics, which will be linked

to the EQUIPMENT table. Note: the three diagrams shown above are included in the repository and may be found [here](#).

API/interface contract, Module-Boundary Diagram

Design Classes

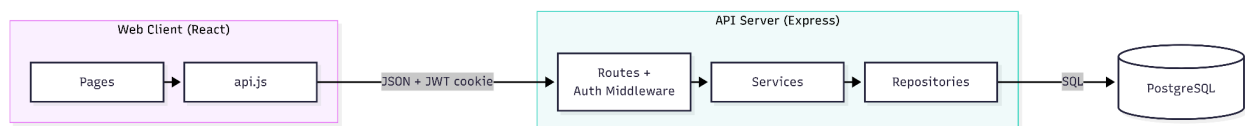
Benchmark's server is split into three layers of classes. The Route Layers receive the requests and then pass them to a service. The Service Layer contains the business logic. For example, it checks things like who can access that service and how long a reservation request is. The repositories are the classes that represent the tables in the database.

API/Interface Contract

The client and server communicate through a JSON REST API. These are the endpoints:

- **authentication:** /api/auth/register, /login, /logout
- **labs and members:** /api/labs, /api/labs/:id/members
- **equipment:** /api/labs/:id/equipment
- **reservations:** /api/labs/:id/equipment/:eid/reservations

Module-Boundary Diagram



Architecture Decision Record

An ADR was created for the use of JWTs to manage the user's role/identity after they have signed in. This is an important document as it defines authentication for the user. This document can be found in the repository [here](#). A copy of it is provided below:

ADR001 Use JSON web tokens(JWT) to handle authentication.

Status: Accepted

Context:

BenchTime must identify the user(as a member or manager) on every API request to understand what endpoints they can access(see FR-3). For example, a user that is a member must only be able to get their own labs and we must guarantee that this authentication cannot be forged by the client.

Decision:

BenchTime will use stateless JWTs for user authentication. `requireAuth`(found in `middleware/auth.js`) will verify the user's access cookie and sets the `req.user = {id, role}`. This means that the JWTs will act as the only source of the user identity in the server and no endpoint exposed by the server will accept a user id/password as a parameter. The server will also not store any state for the user since JWTs are stateless.

Alternatives Considered

Server-side sessions:

Rejected for now but could be revisited at a later milestone. This would require a session store that has not been created and would further complicate the database. A server-side session could also introduce latency(due to extra database queries). JWTs were considered a better fit for the MVP as they are easier to implement.

Using HTTP Basic Auth:

Rejected because it could introduce some vulnerabilities if not encrypted properly and only allows for credentials on each request. The issue here is that the server also requires a role to be associated with each set of credentials so it can properly identify the user. It is also better if the user can send their unique id on every request instead of their username/password.

Consequences For BenchTime

Positives:

- The API does not require any states which keeps the database simple and reduces latency from extra query operations.
- The user role can only come from a signed token which makes certain queries, like those done to request the labs associated with a certain manager, easier to handle as they do not need to perform their own authentication or role identification.

Negatives:

- The JWTs are signed but not encrypted. This means that anyone with the token can see the payload. This also means that care must be taken to not expose anything besides the user's id and their role.

- Two secrets must exist in the deployment environment for the JWT tokens(access and refresh, see `.env.example`). If either of these are changed, then everyone with those tokens will be signed out.

Component Design

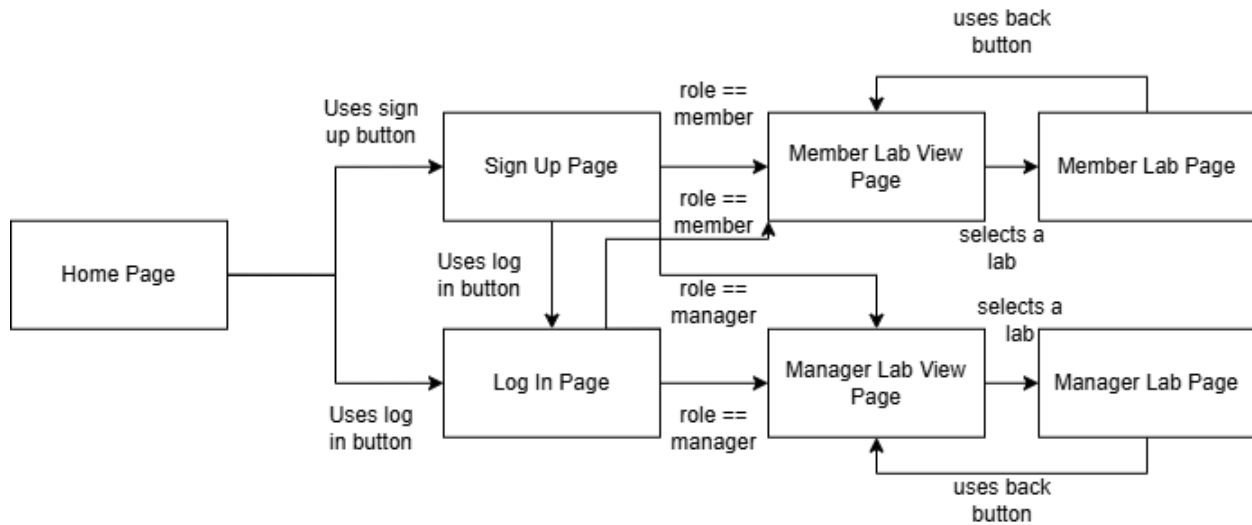
Use Case 1: Add Member to a Lab

When a lab manager opens up a lab, they can see a list of current members. They can use the AddMemberForm where the manager can type in the members username. When they submit the form, that member is added to the lab. The server can respond with a few scenarios: the user does not exist, the manager tries to add themselves, the account belongs to another lab manager. If the member is successfully added, then that member shows up in the member list immediately.

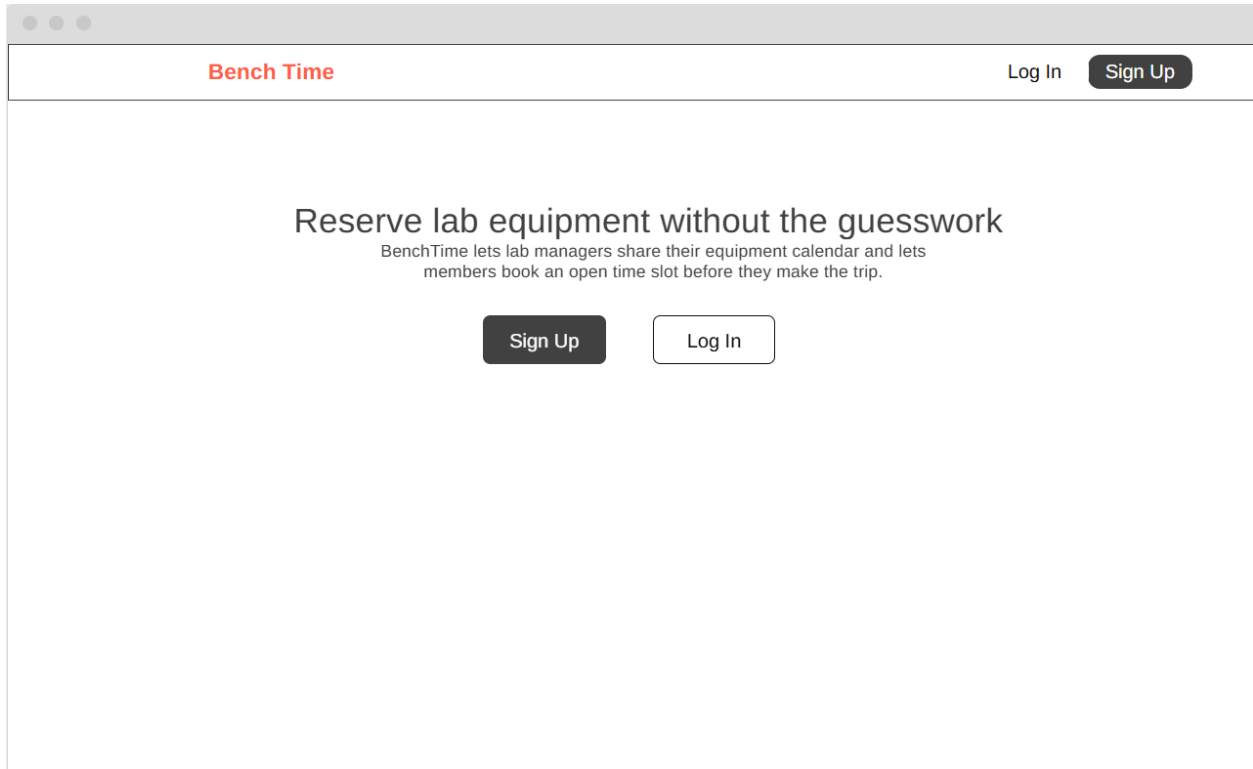
Use Case 2: Reserve Equipment

On the lab page, a member can view all the equipment with a 'Reserve' button next to them. When they press the 'Reserve' button, it opens a reservation form which has a list of upcoming reservations in chronological order. The form also has inputs for the date, the start time, and the end time. To help with the user experience, when a start time is chosen, it automatically selects an hour later for the end time. Once the user submits this reservation, it checks it on the backend for any conflicts or issues. If it succeeds, the reservation shows up in the reservation list for the user.

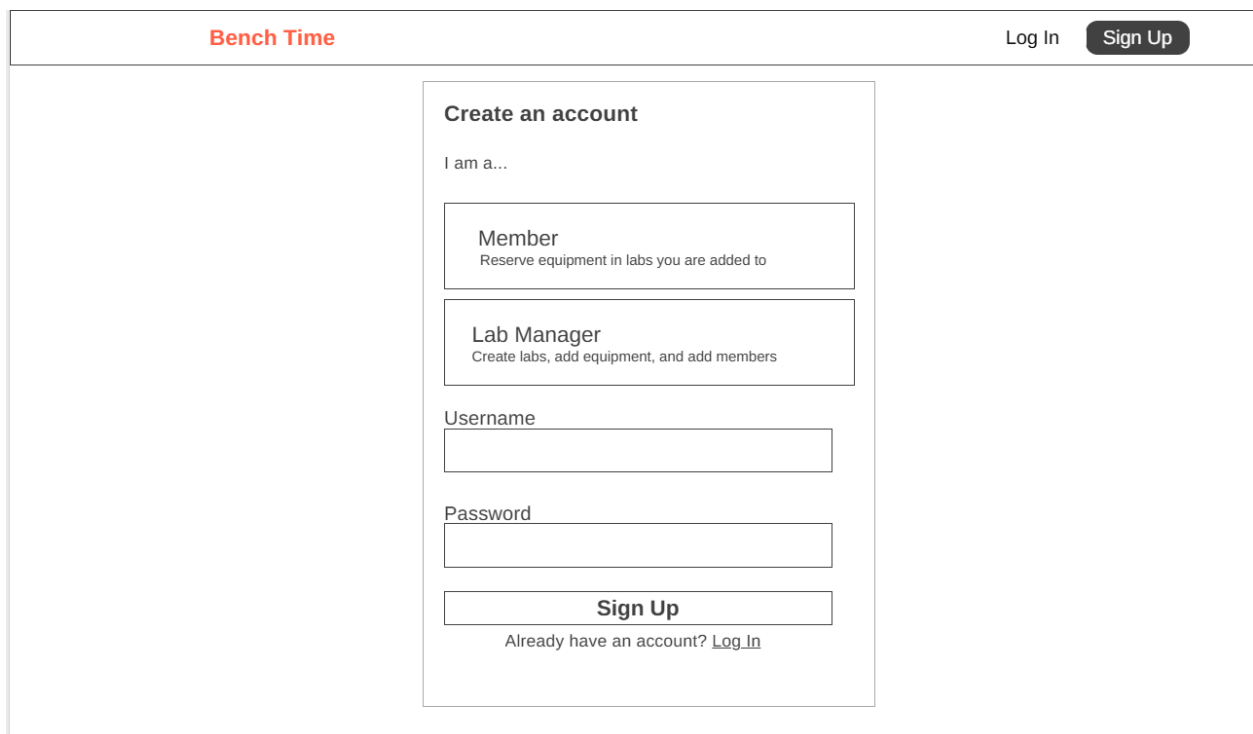
UI/UX Wireframes



The diagram above gives a rough overview of how a user can navigate the webpages provided by BenchTime. The images below are wireframes of the webpages used in BenchTime. Note that the actual website and the wireframes different a little bit in terms of font size and color but the overall structure is the same.



This wireframe shows the home page that all users will see when entering BenchTime without a valid JWT.



This wireframe shows the sign up page that all users will see when they want to create an account.

The wireframe shows a sign up page for 'Bench Time'. At the top, there is a header bar with the 'Bench Time' logo on the left and 'Log In' and 'Sign Up' links on the right. The 'Sign Up' link is highlighted with a dark background. Below the header, the main content area is centered and contains a 'Log in' form. This form includes a 'Username' label and a text input field, a 'Password' label and a text input field, a 'Log In' button, and a link for users without an account: 'No account? [Sign up](#)'.

This wireframe shows the log in page that all users will see when they want to log into BenchTime.

Bench Time
[Username] [Role] Log Out

Your Labs
Cancel

Lab Name

Create Lab

[Lab Name 1]
[Member count, Amount of Equipment]
Creation Date

[Lab Name 2]
[Member count, Amount of Equipment]
Creation Date

This screenshot shows the page where managers and members can view their labs. Note: Only the managers can create/delete labs so for members only the labs they have been added to are visible.

Bench Time
[Username] [Role] Log Out

[<- Back to your labs](#)
[Lab Name]
Managed by [Manager Name]

Equipment
Add Equipment

[Equipment Name]
Reserve X

Members
Add Member

[Member Name]
[Date Added] X

This screenshot shows the page for a lab/equipment calendar. Members would not see the buttons to add/remove equipment/members nor the list of members, only managers would.

Otherwise, the pages are the same.

Bench Time

[Username] [Role] Log Out

[<- Back to your labs](#)

[Lab Name]

Managed by [Manager Name]

Equipment

Add Equipment

[Equipment Name]

Hide schedule X

Upcoming reservations

[Reservation date range]

[Reservation Holder Name] Cancel

Reserve [Equipment Name]

Start

End

[mm/dd/yyyy] 

[mm/dd/yyyy] 

Reserve

Members

Add Member

[Member Name]

[Date Added] X

This screenshot shows the same page after the “Reserve” button is clicked. Any user can reserve equipment and can see this.

Bench Time

[Username] [Role] Log Out

[<- Back to your labs](#)

[Lab Name]
Managed by [Manager Name]

Equipment

Cancel

Equipment Name

Description

Add Equipment

This screenshot shows the lab page once a manager hits the “Add equipment” button.

133

Bench Time

[Username] [Role] Log Out

[<- Back to your labs](#)

[Lab Name]
Managed by [Manager Name]

Members

Cancel

Member username

Add Member

This screenshot shows the lab page once a manager hits the “Add member” button.

Responsive Design

Benchmark is built with a mobile first methodology. This ensures that everything fits on a mobile screen and a desktop screen. By building mobile first, it prevents a situation of having a design that fits on a desktop and having to condense it down to a mobile view. For the date and time picker, we used the browser's built in selector since that is what the user is used to. For cross platform concerns, we tested it on Chrome, Firefox, and Edge to make sure it worked on the most popular browsers.

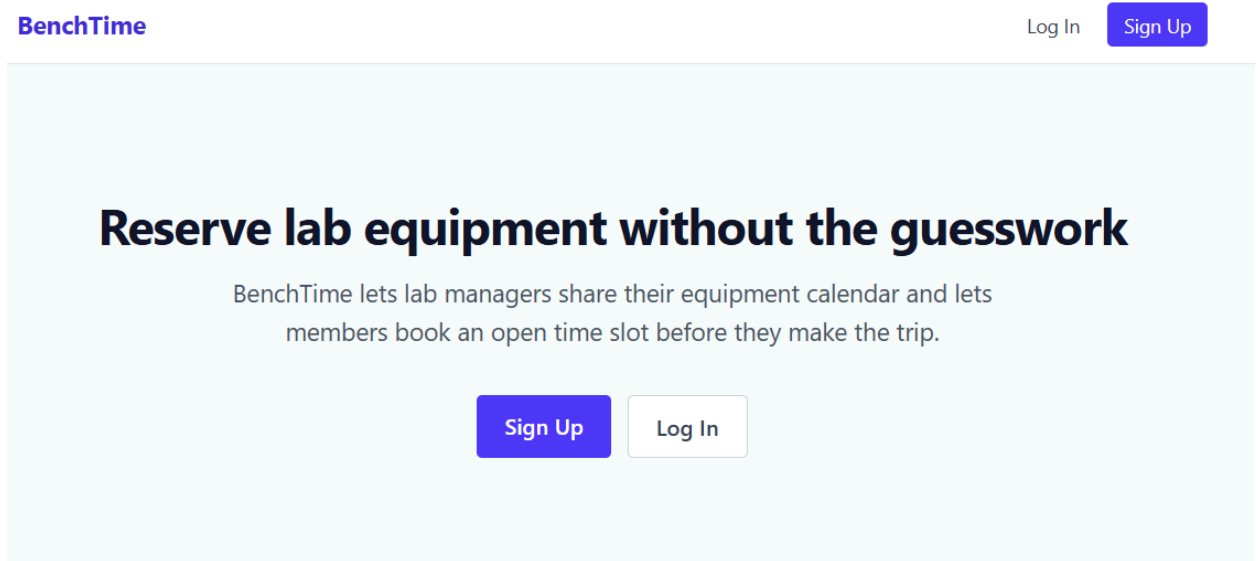
Design Pattern

One design pattern that we chose for BenchTime is the repository design pattern. This pattern provides an abstraction layer between the database and the rest of the server logic. For example, all of the services running on the server must use the repository to access the database and should not query the database directly. This design pattern is extremely useful for a couple of reasons. First, the abstraction layer that it provides allows changes to the database without making changes to the services layer. This is nice because it allows the services layer to maintain a high amount of cohesion and very low coupling. Second, it allows database errors to be standardized. The question of what should happen when a query results in an error, such as a user id not existing, can be standardized all within the repositories. No other layer has to worry about doing this. Finally, the repository pattern provides a centralized location for all queries which increases maintainability and readability.

MVP Features/Demonstration

List of features completed for MVP:

- Sign up and log in as either a member or a lab manager. Passwords are hashed and usernames have to be unique.



Create an account

I am a...

☒ **Member**
Reserve equipment in labs you are added to.

☐ **Lab Manager**
Create labs, add equipment, and add members.

Username

JesseMember

3–32 characters: letters, numbers, _ . -

Password

.....



At least 8 characters.

Sign Up

Already have an account? [Log in](#)

Log in

Username

Password

Log In

No account? [Sign up](#)

- Create and delete labs as a manager.

BenchTime

JesseManager · Lab Manager

Log Out

Your labs

Cancel

Lab name

TestLab

At most 30 characters.

Create Lab

Test1

1 member · 1 piece of equipment

Created 9/30/2026 ×

Test

0 members · 0 pieces of equipment

Created 9/30/2026 ×

- Add and remove members to a lab by username. Once a member is added, the lab shows up on their home page.

BenchTimeJesseManager · Lab ManagerLog Out

[← Back to your labs](#)

Test1

Managed by **JesseManager** (you) · Created 9/30/2026

Equipment

Add equipment

Test	Reserve	×
Test		

Members

Cancel

Member username

The username the member signed up with.

Add Member

JesseMember	Added 9/30/2026	×
-------------	-----------------	---

- Add and remove equipment in a lab.

BenchTimeJesseManager · Lab ManagerLog Out

[← Back to your labs](#)

Test1

Managed by **JesseManager** (you) · Created 9/30/2026

Equipment

Cancel

Equipment name

Description (optional)

Add Equipment

Test

Test

Reserve ×

Members

Add member

JesseMember

Added 9/30/2026 ×

- View the equipment in any lab you own or were added to.
- Reserve equipment for up to 6 hours and see everyone's upcoming reservations for it.

Overlapping reservations get rejected and the user gets told why.

BenchTimeJesseManager · Lab ManagerLog Out

[← Back to your labs](#)

Test1

Managed by **JesseManager** (you) · Created 9/30/2026

Equipment

Add equipment

TestHide schedule ×

Test

Upcoming reservations

Oct 1, 2026, 4:31 PM – 8:31 PM	You	Cancel
Oct 1, 2026, 8:31 PM – 9:31 PM	You	Cancel
Oct 3, 2026, 8:31 PM – 9:31 PM	You	Cancel

Reserve Test

Start

10/01/2026 05:32 PM

📅

End

10/01/2026 09:32 PM

📅

Reservations can be up to 6 hours long.

That time overlaps an existing reservation. Pick a different time.

Reserve

Members

Add member

JesseMember

Added 9/30/2026 ×

Authentication

Because BenchTime uses accounts, we need a simple way of authenticating users. This is done with a simple username/password system. When signing up, a user is asked to provide a username, a password, and to select what role they would like(member or manager). When logging in, they must provide this username and password to be authenticated. The server stores every user as a unique id which can be used as the primary/foreign key for tables inside of the SQL database. Since the system is built using JWT, a client will get a token every time they are authenticated. To initiate certain requests, the client must provide this token to authenticate their unique id and their role as members and managers have different access privileges.

Github Repository

- The github repository can be found here:
<https://github.com/luke-mca/BenchTime/tree/main>
- Note: Instructions to install the software, migrate the database, and what to verify are included in the repository README. The contribution statements are also included in the README.